

Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

Esta unidad cubre las actividades específicas de SQA (Aseguramiento de la Calidad del Software). La **Calidad de Proceso** se refiere a cómo se construye (ej. usando Inspecciones, CMMI, ISO 9001) y la **Calidad de Producto** se refiere a qué se construye (ej. usando testing, midiendo defectos).

Planificación de pruebas - Niveles y tipos de pruebas

- **Planificación:** El **Plan de Pruebas** (basado en el estándar **IEEE 829**) es el documento estratégico. Define:
 - **Alcance:** Características a probar y *fuera* de alcance.
 - **Estrategia de Prueba:** El enfoque general (ej. basado en riesgos, basado en requisitos).
 - **Recursos:** Hardware, software y personal (roles y responsabilidades).
 - **Cronograma:** Cuándo ocurrirán las actividades de prueba.
 - **Entorno de Prueba:** Especificación del hardware/software/datos necesarios.
 - **Criterios de Entrada:** Qué debe estar listo para *iniciar* las pruebas (ej. "Build de CI aprobado").
 - **Criterios de Salida/Suspensión:** Cuándo se considera *finalizada* la prueba (ej. "95% de casos aprobados, 0 defectos críticos").
 - **Entregables:** (ej. Casos de prueba, informes de defectos, informe final).
 - **Riesgos y Contingencias:** Qué puede salir mal.
- **Niveles de Prueba (Cuándo):** Definen la granularidad.
 - **Pruebas Unitarias:** Verifican componentes individuales (funciones, clases) de forma **aislada** (usando *Mocks* y *Stubs*). Son Caja Blanca, escritas por desarrolladores.
 - **Pruebas de Integración:** Verifican la **interacción y comunicación** entre componentes o sistemas (ej. ¿La API llama correctamente a la base de datos?).
 - **Pruebas de Sistema:** Verifican el sistema completo e integrado contra los **requisitos funcionales y no funcionales**. Son Caja Negra y simulan el uso final.
 - **Pruebas de Aceptación (UAT):** Realizadas por el cliente o usuarios finales para validar que el sistema cumple con sus **necesidades de negocio** y es "apto para su uso". (Tipos: Alpha [internas], Beta [externas]).
- **Tipos de Prueba (Qué):** Definen el objetivo.
 - **Funcionales:** Validan *qué* hace el sistema.
 - **Smoke Testing (Prueba de Humo):** Prueba rápida y superficial para verificar que las funcionalidades principales no estén rotas. Se usa para aceptar o rechazar un build.
 - **Regression Testing (Prueba de Regresión):** Asegura que los *nuevos cambios* no hayan roto funcionalidades *existentes*. Es un candidato ideal para la automatización.
 - **No Funcionales:** Validan *cómo* funciona el sistema ("ilities").
 - **Rendimiento (Performance):** Incluye Pruebas de **Carga** (comportamiento bajo carga esperada), **Estrés** (comportamiento bajo

carga extrema, buscando el punto de ruptura) y **Escalabilidad** (cómo responde el sistema al añadir más recursos).

- **Seguridad:** Busca vulnerabilidades (ej. Inyección SQL, XSS).
- **Usabilidad:** Evalúa la facilidad de uso y la experiencia de usuario (UX/UI).

Técnicas y herramientas para probar software (Testing Dinámico)

Son los métodos sistemáticos para *diseñar* casos de prueba efectivos.

- **Técnicas de Caja Negra (Basadas en especificación):** Se diseñan pruebas sin conocer el código, basándose en los requisitos.
 - **Particiones de Equivalencia:** Reduce el número de casos de prueba agrupando entradas que deberían ser tratadas igual. (Ej. para "edad 18-65", las particiones son <18, 18-65, >65).
 - **Análisis de Valores Límite (BVA):** Se enfoca en los "bordes" de las particiones, donde ocurren la mayoría de los errores. (Ej. 17, 18, 65, 66).
 - **Tablas de Decisión:** Para lógica de negocio compleja con múltiples condiciones (ej. "Si es cliente VIP Y compra > \$1000 O usa cupón...").
 - **Transición de Estados:** Para sistemas que tienen "memoria" o estados (ej. un cajero automático: "Inactivo" -> "Autenticado" -> "En Transacción").
- **Técnicas de Caja Blanca (Basadas en estructura):** Se diseñan pruebas conociendo el código, para asegurar que se recorren los caminos lógicos.
 - **Cobertura (Code Coverage):** Mide qué porcentaje del código se ejecutó.
 - **Cobertura de Sentencias (Statement):** ¿Se ejecutó cada línea?
 - **Cobertura de Decisión/Rama (Branch):** ¿Cada **IF** se evaluó como **true** y **false**? (Es más rigurosa que la de sentencias).
 - **Cobertura de Camino (Path):** ¿Se probó cada camino lógico posible? (A menudo es inalcanzable por la cantidad de combinaciones).
- **Herramientas:** Selenium, Cypress (Automatización Web); JMeter, Gatling (Rendimiento); Postman, REST Assured (Pruebas de API); Jira, TestRail (Gestión de Pruebas).

Testing agile

El testing ágil cambia fundamentalmente la mentalidad:

- **"Shift-Left" Testing:** Mover la calidad "hacia la izquierda" (más temprano en el ciclo). Los testers no solo encuentran defectos al final, sino que *previenen* defectos colaborando con el PO en la definición de **Criterios de Aceptación** claros *antes* de que comience el desarrollo.
- **Calidad de Todo el Equipo (Whole Team Approach):** La calidad deja de ser responsabilidad de un departamento de QA y pasa a ser responsabilidad de todo el equipo de desarrollo.
- **Pirámide de Pruebas (Test Pyramid):** Es la estrategia de automatización ágil.
 - **Base:** Muchas **Pruebas Unitarias** (rápidas, baratas, aisladas).
 - **Medio:** Menos **Pruebas de Integración/Servicio/API** (más lentas).

- **Cima:** Muy pocas **Pruebas E2E/UI** (lentas, frágiles, costosas).
- **Cuadrantes de Pruebas Ágiles (Brian Marick):** Un modelo para clasificar qué pruebas hacer.
 - **Q1 (Unitarias):** Apoyan al equipo, guiadas por tecnología.
 - **Q2 (Aceptación):** Apoyan al equipo, guiadas por negocio (ATDD, **BDD - Behavior-Driven Development**).
 - **Q3 (Exploratorias):** Critican el producto, guiadas por negocio (Pruebas manuales de usabilidad, UAT).
 - **Q4 (No Funcionales):** Critican el producto, guiadas por tecnología (Rendimiento, Seguridad).
- **Testing Exploratorio:** Es una técnica manual formal (no es "probar al azar") donde el aprendizaje, diseño de la prueba y ejecución ocurren simultáneamente. Se gestiona en "Sesiones" (Session-Based Test Management).

Técnicas y Herramientas para la realización de revisiones técnicas del software (Testing Estático)

Son la verificación de artefactos (requisitos, diseño, código) *sin ejecutarlos*.

- **Técnicas (el proceso):**
 - **Revisiones Informales:** (Pair Programming, revisión por pares).
 - **Recorridos (Walkthroughs):** El **Autor** presenta el artefacto a sus pares. El objetivo es **educar**, validar el enfoque y ganar consenso. Es semi-formal.
 - **Inspecciones:** Es la técnica **más formal** y rigurosa. Su único objetivo es **encontrar defectos**.
 - **Roles:** Requiere roles definidos: **Moderador** (lidera el proceso, no es el autor), **Autor**, **Lector** (presenta el artefacto), **Registrador** (anota los defectos).
 - **Fases:** Sigue un proceso estricto: 1. Planificación, 2. **Preparación Individual** (donde cada inspector revisa el material *antes* de la reunión usando checklists), 3. Reunión de Inspección (solo se registran defectos, **no se discuten soluciones**), 4. Corrección (Rework) y 5. Seguimiento (Follow-up).
- **Herramientas (el soporte):**
 - **Checklists (Listas de Verificación):** Guían la revisión manual para asegurar que se cubran errores comunes y estándares.
 - **Análisis Estático (SAST - Static Application Security Testing):** Herramientas (ej. **SonarQube**, ESLint) que analizan automáticamente el código fuente en busca de:
 - **"Code Smells" (Olores de código):** Síntomas de un mal diseño (ej. "Método Largo", "Clase Dios").
 - **Violaciones de Estilo:** Código que no sigue las guías.
 - **Errores Potenciales:** (ej. variables no inicializadas, código muerto).
 - **Vulnerabilidades de Seguridad:** Patrones de código conocidos que llevan a riesgos (ej. inyección SQL).